AZIMI INNOVATION LAB — SESSION 01

SESSION DATE: 20 OCTOBER 2026

مخاطب، دامنه و جایگاه جلسه

مخاطب: این جلسه برای آموزش معماری و پیاده‌سازی کنترل‌شده سیستم‌های ایجنتی به مهندسان نرم‌افزار طراحی شده است. مسیر آموزش بر یک مسئله واقعی در سامانه موجود (Brownfield)، توضیح مرحله‌به‌مرحله مفاهیم، مثال قابل بررسی و تمرین عملی تکیه دارد. مفاهیم Coding Agent، Multi-Agent، Guardrails و کنترل دسترسی در ارتباط با همان سناریو آموزش داده می‌شوند.

مسئله مهندسی: در یک سامانه موجود (Brownfield)، خروجی یک Coding Agent یا عامل تخصصی تنها «پیشنهاد تغییر» است؛ نه مجوز اعمال Patch، اجرای Migration، تغییر تنظیمات Production یا دسترسی به داده Tenant دیگر. نقطه اتصال این جلسه به معماری Multi-Agent و Guardrails چندلایه، قرارداد مشخص بین Proposal، Approval و Execution است.

جایگاه نمونه‌ها: قطعات ارجاع‌شده از مخازن متعلق به Amin Azimi به‌عنوان نمونه‌های کد پروژه‌ای و شواهد منشأ استفاده می‌شوند؛ کد آموزشی مستقل این جلسه یک Educational Adaptation است و ادعای استقرار تجاری یا تضمین امنیت Production ندارد.

مسیر پیشنهادی ادامه آموزش (نه برنامه تصویب‌شده): معماری و تفکیک مسئولیت عامل‌ها ← طراحی Specialized Coding Agents و تنظیم Capability/Tool Policy ← ادغام کنترل‌ها در کد Brownfield و آزمون‌های امنیتی/بازگشت.

AZIMI INNOVATION LAB — SESSION 01 | INSTRUCTOR ONLY

فهرست مطالب — ۲۴ بخش

فهرست پیونددار ۲۴ بخش همراه با شماره صفحه؛ شماره‌ها از صفحه‌آرایی همین نسخه محاسبه می‌شوند.

01	مرز بین پیشنهاد، تأیید و اجرا	3
02	نقشه تدریس ۹۰ دقیقه‌ای	3
03	پیش‌نیازها و معیار موفقیت	5
04	مدل سه‌مرحله‌ای با مثال ثابت	6
05	دو خانواده نتیجه که نباید مخلوط شوند	6
06	معماری و مرز اعتماد	7
07	تهدیدها، کنترل‌ها و محدودیت‌های باقیمانده	8
08	سناریوی ثابت و وضعیت اولیه	9
09	سه مسیر خروجی	9
10	قرارداد پیشنهاد (Proposal Contract)	10
11	Digest یکسان برای محتوای یکسان	12
12	اعتبارسنجی ورودی در Python	13
13	Policy Gate: کنترل‌هایی که هیچ‌کدام اختیاری نیستند	14
14	ترتیب تصمیم Gate و مسئله Revision	15
15	اجرای تراکنشی SQLite ـ قسمت اول	15
16	مصرف تأیید، ثبت Audit و نتیجه اجرا ـ قسمت دوم	16
17	مرز تصمیم: چرا ALLOW به‌معنای اجرا نیست؟	20
18	سه سناریوی قابل تدریس و نمونه واقعی قابلیت Agent	24
19	شواهد چه چیزی را ثابت می‌کنند؟	25
20	تمرین کاغذی ۱۵ دقیقه‌ای، بدون نیاز به دانلود	27
21	پانزده دقیقه دوم تمرین عملی	28
22	پاسخ‌نامه محرمانه و شیوه نمره‌دهی	29
23	جمع‌بندی، منابع و جلسه بعد	29
24	مالکیت، مجوز تدریس و منشأ محتوا	31

بخش 01 از ۲۴ — مرز بین پیشنهاد، تأیید و اجرا

هدف مدرس: دانشجو بتواند در یک جمله بگوید چرا پیشنهاد Agent مجوز اجرا نیست.

فرض کنید Agent تشخیص می‌دهد تعداد تلاش مجدد در یک سرویس کم است و پیشنهاد می‌دهد `retry_limit` از ۳ به ۵ برسد. Agent می‌تواند چنین پیشنهادی را توصیف کند، اما مجاز نیست با تولید متن «Approved» برای خودش اختیار بسازد. تأیید باید از شخص یا سرویس دارای اختیار و از مسیری مورد اعتماد بیاید. حتی پس از تأیید نیز اجرا باید شرایط را مستقل دوباره بررسی کند.

سه واژه اصلی: `PROPOSAL` توصیف تغییر؛ `APPROVAL` اجازه مشروط برای همان تغییر مشخص؛ `EXECUTION` تلاش کنترل‌شده برای اعمال تغییر. در این جلسه، خروجی Gate با نتیجه واقعی اجرا تفاوت دارد. تمرین: از فراگیر بپرسید اگر Agent بگوید «خودم تأیید کردم»، کدام مرز اعتماد شکسته می‌شود؟

دامنه: هدف جلسه خواندن و تحلیل کد چاپ‌شده و حل تمرین کاغذی است؛ ساخت سامانه اجرایی جزو الزام تمرین نیست.

بخش 02 از ۲۴ — نقشه تدریس ۹۰ دقیقه‌ای

جدول اجرایی مدرس — ۹۰ دقیقه دقیق، با خروجی قابل سنجش

تمرین‌های A–F دقیقاً ۳۰ دقیقه از زمان کلاس را تشکیل می‌دهند؛ Task G فقط پس از کلاس است. مرور کد و کارگاه Brownfield گزینشی است، نه پیاده‌سازی کامل زنده.

بازه (دقیقه)	بخش‌ها	فعالیت	شاهد قابل ارزیابی
۰–۸	01–03	تشخیص مرز اختیار و ثبت دو پیش‌نیاز عملی	توضیح یک failure mode
۸–۲۰	04–06, 08–09	رسم Proposal → Approval → Execution و مرزهای اعتماد	نمودار سه‌مرحله‌ای با مالک هر تصمیم
۲۰–۳۲	10–12	تحلیل Contract، canonical digest و ورودی نامعتبر	فهرست فیلدهای bindشده و یک tamper case
۳۲–۴۵	13–14, 17	ترتیب ارزیابی Gate برای کنترل Tenant، Scope، Revision و Replay	جدول تصمیم با علت و اولویت کنترل
۴۵–۵۵	15–16	Atomicity، خطا و تفاوت ALLOW با Commit	توضیح rollback و نقطه commit
۵۵–۶۰	کارگاه بین 17 و 18	انتخاب محل تزریق Gate در Backend موجود	نقشه مهاجرت سه‌گامی بدون side effect
۶۰–۷۵	20 / A–C	تحلیل فردی و دفاع از تصمیم	۳ پاسخ مستدل در Worksheet
۷۵–۹۰	21 / D–F; 19, 23	تحلیل Replay/Revision، تحویل و جمع‌بندی ۲ دقیقه آخر	جدول سه سناریو + دو محدودیت طراحی

ماتریس هدف ← فعالیت ← سنجش یادگیری

هدف طراحی	شاهد	ارزیابی
طراحی مرز اعتماد و تعیین مسئولیت	نمودار مسیر درخواست	تمرین A و توضیح مرز اعتبار
طراحی اتصال تأیید به نسخه دقیق پیشنهاد	Contract + Digest + Revision	Task B–D با علت تصمیم
طراحی مسیر اجرای امن و بازیابی خطا	Replay و Atomicity	Task E–F و دو محدودیت سیستم
تطبیق کنترل با کد موجود	نقطه تزریق Gate و مسیر rollout	نقشه Brownfield کوتاه؛ ارزیابی تکمیلی

یادداشت مدرس: هر مفهوم را با سناریوی ثابت تغییر کنترل‌شده `retry_limit` آغاز کنید؛ ابتدا نتیجه را از فراگیر پیش‌بینی بخواهید، سپس دلیل تصمیم Gate را با مثال و کد نشان دهید. بخش‌های ۷، ۱۸، ۲۲ و ۲۴ مرجع تکمیلی‌اند؛ پاسخ‌نامه بخش ۲۲ صرفاً برای مدرس مجاز است. در ۲ دقیقه پایانی، تفاوت تصمیم Gate و نتیجه اجرای واقعی را دوباره بررسی کنید.

۱۰–۰ دقیقه: با یک دستور ناامن Agent شروع کنید: «مقدار را تغییر بده، تأیید لازم نیست». از فراگیر بپرسید چرا متن Agent مرجع اختیار نیست.

پرسش مدرس: «اگر Agent بعد از تأیید مقدار را تغییر دهد، دقیقاً چه چیزی جلوی اجرا را می‌گیرد؟» پاسخ مورد انتظار: اتصال تأیید معتبر به Digest کانونیکال پیشنهاد و بررسی دوباره آن در زمان اجرا. صرف گفتن «Agent دستور را رعایت می‌کند» پاسخ پذیرفتنی نیست.

محرمانگی: این نسخه حاوی راهنمای مدرس و پاسخ‌نامه است؛ نباید به فراگیر تحویل یا ارسال شود.

بخش 03 از ۲۴ — پیش‌نیازها و معیار موفقیت

خروجی‌های قابل سنجش: در پایان جلسه شرکت‌کننده باید بتواند (۱) مرز اعتماد و نقاط Side Effect یک جریان Coding Agent را روی دیاگرام مشخص کند؛ (۲) قرارداد Proposal/Approval/Execution را با Tenant، Scope، Revision و Digest طراحی و توجیه کند؛ (۳) دست‌کم سه سناریوی سوءاستفاده شامل Replay، تغییر وضعیت پس از Approval و Cross-Tenant را با تست قابل تکرار رد کند؛ و (۴) محل اتصال Gate به یک مسیر اجرایی موجود را همراه با مسیر Rollback پیشنهاد دهد.

پیش‌نیاز: توانایی خواندن Python یا TypeScript پایه، درک شیء JSON و آشنایی با مفهوم تراکنش پایگاه داده. دانش پیشرفته رمزنگاری لازم نیست.

معیار ارزیابی طراحی: شرکت‌کننده باید علاوه بر تشخیص مراحل Proposal، Approval و Execution، یک قرارداد قابل‌اعتبارسنجی برای Revision، Scope، Tenant و Digest طراحی کند؛ آزمون‌های Replay،

Cross-Tenant و Stale Revision را تعریف کند؛ و محدودیت‌های نسخه آموزشی و مسیر Rollback را مستند سازد.

رفع سوءبرداشت: Digest امضای دیجیتال نیست و هویت تأییدکننده را به‌تنهایی اثبات نمی‌کند. Digest فقط در مقایسه با مقدار نگهداری‌شده نزد مرجع معتبر می‌تواند تغییر محتوای پیشنهاد را آشکار کند. پرسش: اگر Agent بتواند هم پیشنهاد و هم Digest ذخیره‌شده را تغییر دهد، چه تضمینی باقی می‌ماند؟

بخش 04 از ۲۴ — مدل سه‌مرحله‌ای با مثال ثابت

Proposal: «برای tenant-01، کلید `retry_limit` در هدف منطقی `app-config.json` از ۳ به ۵ تغییر کند؛ Revision مورد انتظار ۳ است.» این صرفاً داده‌ای از منبع غیرمورد اعتماد است.

Approval: مرجع معتبر پیشنهاد `P-001` را با Digest مشخص، Tenant و هدف معین، اختیار تأییدکننده و زمان انقضای معتبر تأیید می‌کند. Agent نباید قادر باشد شناسه تأییدکننده یا رکورد مرجع را جعل کند.

Execution: هنگام اجرای درخواست، سامانه همان شرایط را دوباره بررسی می‌کند. فقط در صورت مجازبودن و سازگاری Revision، تراکنش محلی می‌تواند وضعیت را تغییر دهد و تأیید را مصرف کند. تمرین: بپرسید اگر تأیید دیروز معتبر بوده اما امروز منقضی شده، آیا پیشنهاد هنوز قابل اجراست؟ پاسخ: خیر.

بخش 05 از ۲۴ — دو خانواده نتیجه که نباید مخلوط شوند

تصمیم **Gate**: `ALLOW` یعنی اکنون شرایط تلاش برای اجرا برقرار است؛ `DENY` یعنی شرط اختیار یا Policy برقرار نیست؛ `REVALIDATION_REQUIRED` یعنی وضعیت قدیمی شده و باید دوباره ارزیابی شود.

نتیجه اجرا: `SUCCEEDED` یعنی Commit یا اثر تعریف‌شده کامل شده؛ `FAILED` یعنی شکست مشخص رخ داده؛ `UNKNOWN` یعنی معلوم نیست اثر واقعاً اتفاق افتاده یا نه. قاعده: `SUCCEEDED ≠ ALLOW`.

سناریوی مدرس: Gate اجازه می‌دهد، اما سرویس خارجی بعد از ارسال درخواست قطع می‌شود. بدون شواهد کافی، نتیجه ممکن است `UNKNOWN` باشد. تکرار کورکورانه درخواست خطرناک است و به سازوکار reconciliation و idempotency نیاز دارد. نمونه SQLite سند اصلی، ماشین وضعیت کامل برای اثرات خارجی نیست.

بخش 06 از ۲۴ — معماری و مرز اعتماد

مسیر آموزشی: `Agent/LLM → Proposed action (untrusted) → Schema + canonical digest → Trusted approval store → Policy/Tenant/Approver/Expiry gate → Revision check → Atomic local state update + approval consumption + audit → Execution result`.

Agent فقط پیشنهاد را می‌سازد. Schema و Digest شکل و یکپارچگی درخواست را کنترل می‌کنند؛ اختیار از مخزن تأیید و Policy مورد اعتماد می‌آید. قبل از تغییر وضعیت، Revision فعلی باید بررسی شود. در مثال SQLite، تغییر وضعیت، مصرف تأیید و ثبت Audit در یک تراکنش محلی انجام می‌شوند.

فرض اعتماد: یکپارچگی پایگاه داده و حفاظت از رکوردهای تأیید، پیش‌فرض محیطی است؛ متن مدل جایگزین این کنترل‌ها نیست. مرز ادعا: اتمیک‌بودن تراکنش SQLite به معنی اتمیک‌بودن عملیات فایل یا سرویس خارجی نیست.

بخش 07 از ۲۴ — تهدیدها، کنترل‌ها و محدودیت‌های باقیمانده

دست‌کاری **Payload**: مقایسه Digest؛ مشروط به حفاظت از Digest معتبر. اشتباه **Tenant**: جست‌وجوی محدود به Tenant؛ مشروط به Policy جداسازی ذخیره‌سازی. تأییدکننده بی‌اختیار: بررسی مرجع اختیار؛ احراز هویت واقعی خارج از این نمونه است.

Replay: مصرف اتمیک تأیید؛ در سند اصلی فقط نمونه محلی SQLite نشان داده می‌شود. **Revision** قدیمی: کنترل Compare-and-Swap؛ وضعیت خارجی ممکن است مستقل تغییر کند. **Crash** میان تغییرات: تراکنش Rollback/Commit؛ آزمون بازیابی Crash در منبع ارائه نشده است.

ارجاع منبع: سند اصلی در این بخش گزیده `core/human_authority.py` از AEGIS Agent و `ApprovalLevel` شامل `AUTO`، `SINGLE_APPROVAL`، `DUAL_APPROVAL` و `DENY` را نقل می‌کند. تمایز: طبقه‌بندی ریسک به‌تنهایی رکورد تأیید متصل به Digest نیست.

SOURCE EXCERPT — PINNED REVISION؛ مجوز و شماره خطوط در حال بررسی: core/human_authority.py · 4f310113c49c

```
class ApprovalLevel(Enum):
    AUTO = auto()
    SINGLE_APPROVAL = auto()
    DUAL_APPROVAL = auto()
    DENY = auto()
```

این قطعه طبقه‌بندی سطح تأیید را نشان می‌دهد، نه تأیید خودکار توسط Agent. AUTO صرفاً تصمیم Policy مورد اعتماد است؛ DUAL_APPROVAL خارج از نمونه تک‌تأییدکننده این جلسه است.

بخش 08 از ۲۴ — سناریوی ثابت و وضعیت اولیه

HYPOTHETICAL SCENARIO: هدف منطقی `app-config.json`، Tenant برابر `tenant-01`، عملیات `set_config`، کلید `retry_limit`، مقدار فعلی ۳، مقدار پیشنهادی ۵ و Revision مورد انتظار ۳.

ابتدا Agent پیشنهاد `P-001` را می‌سازد؛ سپس تأییدکننده معتبر دقیقاً همان پیشنهاد را تأیید می‌کند. زمان درخواست اجرا، سامانه Tenant، اختیار، Digest، انقضا و Revision را دوباره کنترل می‌کند. اجازه نقش (Permission) با تأیید Digest-bound یکسان نیست.

محدودیت مهم: وضعیت آموزشی SQLite صرفاً Revision تنظیمات را شبیه‌سازی می‌کند؛ کد، فایل واقعی `app-config.json` را باز، قفل، بازنویسی یا `fsync` نمی‌کند. بنابراین ادعای جایگزینی اتمیک فایل نادرست است.

بخش 09 از ۲۴ — سه مسیر خروجی

A — VALID: مقدار ۵، Digest یکسان، تأیید معتبر و Revision فعلی ۳. تصمیم Gate برابر `ALLOW` است؛ فقط Commit موفق تراکنش محلی می‌تواند به `SUCCEEDED` منجر شود.

B — TAMPERED: پس از تأیید، مقدار ۵ به ۹ تغییر می‌کند. Digest دیگر منطبق نیست؛ نتیجه `DENY` و بدون تغییر وضعیت است.

C — STALE: پیشنهاد همان است، اما Revision فعلی ۴ شده است. نتیجه `REVALIDATION_REQUIRED` و بدون تغییر وضعیت است. پیشنهاد جدید بر اساس Revision تازه، Digest جدید و تأیید معتبر جدید لازم است؛ تأیید قدیمی نباید استفاده شود.

ارجاع منبع: گزیده `core/authority_matrix.py` در سند اصلی `check_permission` را نشان می‌دهد؛ بررسی نقش در Tenant به‌تنهایی مجوز Digest-bound برای اجرای یک پیشنهاد مشخص نیست. تمرین مدرس: برای هر مسیر ابتدا نتیجه Gate، سپس نتیجه اجرای احتمالی و علت را جداگانه بخواهید.

SOURCE EXCERPT — PINNED REVISION؛ مجوز و شماره خطوط در حال بررسی: core/authority_matrix.py · f845ab702fba

```
def check_permission(self, tenant_id: str, actor: str, permission: str) -> bool:
    with self._lock:
        roles = self._assignments.get(tenant_id, {}).get(actor, [])
        for r in roles:
            role = self._roles.get(r)
            if role and permission in role.permissions:
                EvidenceLedgerSingleton.append_entry(
                    tenant_id=tenant_id,
                    actor="authority_matrix",
                    action="check_permission",
                    payload={"actor": actor, "permission": permission, "result": True},
                )
                return True
        EvidenceLedgerSingleton.append_entry(
            tenant_id=tenant_id,
            actor="authority_matrix",
            action="check_permission",
            payload={"actor": actor, "permission": permission, "result": False},
        )
        return False
```

این تابع فقط مجوز نقش را در Tenant کنترل می‌کند؛ به‌تنهایی جایگزین تأیید متصل به Digest نیست.

بخش 10 از ۲۴ — قرارداد پیشنهاد **(Proposal Contract)**

هدف تدریس: فراگیر بتواند بین «مشخص‌کردن دقیق درخواست» و «مجازبودن اجرای درخواست» تمایز بگذارد.

Agent می‌تواند درخواست تغییر بسازد، اما هنوز اجازه اجرا ندارد. پیش از تصمیم‌گیری باید معلوم باشد چه کسی/کدام درخواست، در کدام **Tenant**، چه عملیاتی را روی چه هدفی، با چه مقدار و بر مبنای کدام **Revision** پیشنهاد کرده است. قرارداد زیر یک نمونه آموزشی محدود است؛ سیاست مجوزدهی کامل نیست.

EDUCATIONAL ADAPTATION — قرارداد داده (Data Contract)؛ تعریف ساختار داده به‌تنهایی مجوز اجرا نیست:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Proposal:
    proposal_id: str
    tenant_id: str
    operation: str
    target: str
    key: str
    value: int
    revision: int
```

شرح برای مدرس: `proposal_id` شناسه پیشنهاد؛ `tenant_id` مرز مستأجر؛ `operation` نوع عملیات؛ `target` مقصد منطقی؛ `key` ویژگی مورد تغییر؛ `value` مقدار درخواستی؛ `revision` نسخه‌ای است که پیشنهاد بر پایه آن ساخته شده است. `frozen=True` انتساب معمول مجدد به ویژگی‌ها را محدود می‌کند، اما اثبات هویت، اعتبار داده یا مجوز اجرا نیست.

سناریوی فرضی: P-001 برای `tenant-01` درخواست می‌دهد `retry_limit` از ۳ به ۵ تغییر کند، با Revision برابر ۳. سؤال مدرس: «اگر فقط مقدار را به ۹ تغییر دهیم، آیا تأیید قبلی همچنان معتبر است؟» پاسخ: خیر؛ پیشنهاد تغییر کرده و باید تطابق Digest و شرایط تأیید دوباره بررسی شوند.

مرز قرارداد: در این جلسه فقط رشته‌های ASCII چاپ‌پذیر و اعداد صحیح در محدوده امن JavaScript پذیرفته می‌شوند؛ این روش یک استاندارد عمومی Canonical JSON نیست.

بخش 11 از ۲۴ — **Digest** یکسان برای محتوای یکسان

هدف تدریس: فراگیر بفهمد چرا تأیید باید به نمایش قطعیِ دقیقِ پیشنهاد متصل شود.

اگر دو نسخه از یک پیشنهاد فقط در ترتیب کلیدهای JSON متفاوت باشند، نباید بی‌دلیل Digest متفاوت بگیرند. برای همین کلیدها مرتب و جداکننده‌ها ثابت می‌شوند. سپس SHA-256 روی بایت‌های خروجی اعمال می‌شود. این Digest باید همراه رکورد تأیید در مخزن مورد اعتماد نگهداری شود؛ Agent نباید بتواند Digest مرجع را تغییر دهد.

EDUCATIONAL ADAPTATION — بازنویسی آموزشی مبتنی بر منبع؛ گزارش آزمون منبع به‌تنهایی به معنی آزمون مستقل این بازنویسی نیست:

```
SAFE_MAX = 9007199254740991

def digest(p: Proposal) -> str:
    validate_contract(p)
    record = {'schema_version': 1, **asdict(p)}
    payload = json.dumps(record, sort_keys=True,
                         separators=(',', ':'),
                         ensure_ascii=True).encode('ascii')
    return hashlib.sha256(payload).hexdigest()
```

شرح خط‌به‌خط: ابتدا `validate_contract` نوع و محدوده فیلدها را کنترل می‌کند؛ سپس `schema_version` به رکورد اضافه می‌شود.
`sort_keys=True` ترتیب کلیدها را تثبیت می‌کند، `separators` فاصله‌های غیرضروری را حذف می‌کند و `ensure_ascii=True` با قرارداد محدود

ASCII هماهنگ است. در پایان، SHA-256 یک Digest هگزادسیمال تولید می‌کند.

وابستگی‌های نمونه: `import hashlib`، `import json` و `from dataclasses import dataclass, asdict`؛ تعریف کامل `Proposal` در بخش ۱۰ و تابع `validate_contract` در بخش ۱۲ است. هش برابر، به‌تنهایی هویت تأییدکننده یا اختیار او را اثبات نمی‌کند.

تمرین کوتاه: مقدار `value` را از ۵ به ۹ تغییر دهید و توضیح دهید چرا تأیید متصل به Digest قبلی دیگر برای پیشنهاد جدید قابل استفاده نیست. این یک پیش‌بینی آموزشی است، نه گزارش اجرای این مثال.

بخش 12 از ۲۴ — اعتبارسنجی ورودی در Python

هدف تدریس: فراگیر تفاوت اعتبارسنجی شکل داده با Policy Gate را توضیح دهد.

پیش از ساخت Digest، باید داده‌هایی که ممکن است مبهم یا خارج از قرارداد باشند رد شوند. در Python، مقدار `True` از خانواده `int` است؛ به همین دلیل استفاده از `type(value) is int` به‌جای پذیرش ساده با `isinstance` در این قرارداد اهمیت دارد.

EDUCATIONAL ADAPTATION — بازنویسی آموزشی مبتنی بر منبع؛ گزارش آزمون منبع به‌تنهایی به معنی آزمون مستقل این بازنویسی نیست:

```
def validate_contract(p: Proposal) -> None:
    if type(p) is not Proposal:
        raise ValueError('Invalid proposal type')
    fields = asdict(p)
    expected = {'proposal_id', 'tenant_id', 'operation',
                'target', 'key', 'value', 'revision'}
    if set(fields) != expected:
        raise ValueError('Invalid field set')
```

```
    for key, value in fields.items():
        if key in ('value', 'revision'):
            if type(value) is not int or not -SAFE_MAX <= value <= SAFE_MAX:
                raise ValueError('Invalid integer')
        elif type(value) is not str or not value or any(
                not 32 <= ord(ch) <= 126 for ch in value):
            raise ValueError('Invalid string')
```

شرح برای مدرس: ابتدا خود شیء باید دقیقاً از نوع `Proposal` باشد. سپس مجموعه فیلدها با قرارداد مقایسه می‌شود. `value` و `revision` باید عدد صحیح واقعی و در محدوده مجاز باشند؛ رشته‌ها باید غیرخالی و فقط از کاراکترهای ASCII چاپ‌پذیر تشکیل شده باشند. در مرحله بعد، **Policy Gate** محدودیت‌های معنایی بیشتری اعمال می‌کند: مقدار `value` بین ۱ تا ۱۰ و `revision` نامنفی باشد.

سه سؤال سنجش: آیا `value=True` پذیرفته می‌شود؟ خیر. آیا یک رشته Unicode خارج از ASCII چاپ‌پذیر پذیرفته می‌شود؟ خیر. آیا عبور از این Validator یعنی درخواست مجاز است؟ خیر؛ تأیید، اختیار، Tenant، Digest، انقضا، Revision و Policy هنوز باید بررسی شوند.

محدودیت امنیتی: این Validator مخصوص یک `dataclass` ثابت است و **Parser** کامل درخواست شبکه محسوب نمی‌شود. نمونه‌های مخزن‌های مرتبط فقط برای توضیح کنترل دسترسی هستند و پیاده‌سازی همین Gate آموزشی نیستند.

بخش 13 از ۲۴ — **Policy Gate**: کنترل‌هایی که هیچ‌کدام اختیاری نیستند

مرجع واحد این چهار بخش: کد آموزشی مستقل `session01_reference.py`؛ این کد نقل مستقیم از GitHub نیست. نمونه

فقط وضعیت محلی SQLite را تغییر می‌دهد و فایل واقعی `app-config.json` را بازنویسی نمی‌کند.

ترتیب **Gate**: ۱. قرارداد و نوع Proposal؛ ۲. allowlist عملیات/کلید/هدف؛ ۳. زمان معتبر؛ ۴. رکورد تأیید محدود به Tenant؛ ۵. شناسه پیشنهاد؛ ۶. Digest؛ ۷. وضعیت APPROVED؛ ۸. انقضا و هدف؛ ۹. اختیار تأییدکننده؛ ۱۰. Revision. اختلاف Revision خروجی `REVALIDATION_REQUIRED` دارد.

مهم: الگوریتم Canonical این دوره سفارشی است و ادعای انطباق با RFC 8785 ندارد. Digest امضا نیست؛ مرجع Digest باید از Agent مستقل باشد.

بخش 14 از ۲۴ — ترتیب تصمیم **Gate** و مسئله **Revision**

سه تصمیم **Gate**: `DENY` برای فقدان اختیار یا عدم انطباق تأیید؛ `REVALIDATION_REQUIRED` برای Revision قدیمی؛ `ALLOW` فقط برای اجازه تلاش اجرای محلی. مصرف‌شدن تأیید (`CONSUMED`) مانع Replay است. برای هر شاخه، نتیجه Gate را جدا از نتیجه اجرا بیان کنید.

SQL allowlist: نام ستون SQL با placeholder پارامتری نمی‌شود؛ بنابراین فقط زوج مشخص `(set_config, retry_limit, app-config.json)` مجاز است. هدف منطقی فایل در این مثال به ستون `config_state.retry_limit` نگاشت می‌شود.

بخش 15 از ۲۴ — اجرای تراکنشی **SQLite** ـ قسمت اول

راه‌اندازی اتصال: `sqlite3.connect(':memory:', isolation_level=None, timeout=5)`، سپس `BEGIN IMMEDIATE`. SQLite در این نمونه نویسنده هم‌زمان واحد دارد؛ `timeout` انتظار برای قفل را

محدود می‌کند. شرط `WHERE revision=?` نیز Compare-and-Swap را آموزش می‌دهد.

Audit: شناسه پیشنهاد، Digest، Tenant، مقدار پیش/پس، Revision پیش/پس، زمان و نتیجه ثبت می‌شود. تلاش ردشده که همراه تراکنش rollback شود ثبت پایدار ندارد؛ ثبت مستقل آن طراحی جداگانه می‌خواهد.

بخش 16 از ۲۴ — مصرف تأیید، ثبت **Audit** و نتیجه اجرا ـ قسمت دوم

رفع تناقض مسیر خطا: خطاهای قبل از Commit موجب `rollback` و سپس `raise` می‌شوند؛ خطا بلعیده نمی‌شود. خطای Commit لزوماً شکست قطعی یا `UNKNOWN` نیست؛ در حالت ابهام باید وضعیت با Reconciliation بررسی شود. `FAILED` و `UNKNOWN` در Enum تعریف شده‌اند، ولی این نمونه فقط موفقیت Commit محلی را `SUCCEEDED` گزارش می‌کند.

کد مرجع کامل و مستقل، **COURSE CODE — TESTED**: ۱۱ آزمون unittest در ۱۴ اکتبر ۲۰۲۶ موفق؛ بدون آزمون Crash Recovery یا فایل واقعی.

```
"""Course-authored reference, NOT a verbatim excerpt from any GitHub repository.
Demonstrates SQLite-local authorization and state transition, not real file I/O.
"""
from dataclasses import dataclass
from enum import Enum
import hashlib, json, sqlite3

SAFE_MAX = 2**53 - 1
class GateDecision(str, Enum):
    ALLOW='ALLOW'; DENY='DENY'; REVALIDATION_REQUIRED='REVALIDATION_REQUIRED'
class ExecutionResult(str, Enum):
    SUCCEEDED='SUCCEEDED'; FAILED='FAILED'; UNKNOWN='UNKNOWN'
@dataclass(frozen=True)
class Proposal:
```

```
    proposal_id:str; tenant_id:str; target:str; operation:str; key:str; value:int; revision:int

def validate(p):
    if not isinstance(p,Proposal): raise ValueError('invalid proposal')
    for x in (p.proposal_id,p.tenant_id,p.target,p.operation,p.key):
        if not isinstance(x, str) or not x:
            raise ValueError('invalid ASCII field')
        if any(ord(c)<33 or ord(c)>126 for c in x):
            raise ValueError('invalid ASCII field')
    if type(p.value) is not int or not 1<=p.value<=10: raise ValueError('invalid value')
    if type(p.revision) is not int or not 0<=p.revision<=SAFE_MAX: raise ValueError('invalid revision')

def digest(p):
    validate(p)
    fields = ('proposal_id', 'tenant_id', 'target',
              'operation', 'key', 'value', 'revision')
    payload = {k: getattr(p, k) for k in fields}
    encoded = json.dumps(
        payload, sort_keys=True, separators=(',', ':'),
        ensure_ascii=True
    ).encode('ascii')
    return hashlib.sha256(encoded).hexdigest()

def connect():
    c=sqlite3.connect(':memory:',isolation_level=None,timeout=5)
    c.executescript('''
    CREATE TABLE config_state(tenant_id TEXT PRIMARY KEY,
        retry_limit INTEGER NOT NULL,
        revision INTEGER NOT NULL);
    CREATE TABLE approvals(approval_id TEXT PRIMARY KEY,
        tenant_id TEXT NOT NULL,
        proposal_id TEXT NOT NULL,
        digest TEXT NOT NULL,
        status TEXT NOT NULL,
        expires_at INTEGER NOT NULL,
        approver TEXT NOT NULL,
        target TEXT NOT NULL);
    CREATE TABLE approver_permissions(tenant_id TEXT NOT NULL,
        approver TEXT NOT NULL,
        operation TEXT NOT NULL,PRIMARY KEY(tenant_id,approver,operation));
    CREATE TABLE audit(id INTEGER PRIMARY KEY,
        approval_id TEXT NOT NULL,
        tenant_id TEXT NOT NULL,
        proposal_id TEXT NOT NULL,
        digest TEXT NOT NULL,
        old_value INTEGER,
        new_value INTEGER,
        revision_before INTEGER,
        revision_after INTEGER,
        at INTEGER NOT NULL,
        result TEXT NOT NULL);
    ''')
    return c
```

```
def seed(c,p,now=1000,expiry=2000,approver='instructor',approval_id='A-001'):
    c.execute('INSERT INTO config_state VALUES (?,?,?)',(p.tenant_id,3,3))
    c.execute(
        'INSERT INTO approvals VALUES (?,?,?,?,?,?,?,?)',
        (approval_id, p.tenant_id, p.proposal_id,
         digest(p), 'APPROVED', expiry, approver, p.target)
    )
    c.execute(
        'INSERT INTO approver_permissions VALUES (?,?,?)',
        (p.tenant_id, approver, p.operation)
    )

def gate(c,p,approval_id,trusted_now):
    try: validate(p)
    except (ValueError,TypeError): return GateDecision.DENY
    if type(trusted_now) is not int or trusted_now<0: return GateDecision.DENY
    allowed = ('set_config', 'retry_limit', 'app-config.json')
    if (p.operation, p.key, p.target) != allowed:
        return GateDecision.DENY
    a = c.execute(
        'SELECT proposal_id,digest,status,expires_at,'
        'approver,target FROM approvals '
        'WHERE approval_id=? AND tenant_id=?',
        (approval_id, p.tenant_id)
    ).fetchone()
    if not a: return GateDecision.DENY
    pid,dig,status,expiry,approver,target=a
    if (pid != p.proposal_id or dig != digest(p)
            or status != 'APPROVED'
            or expiry < trusted_now or target != p.target):
        return GateDecision.DENY
    permission = c.execute(
        'SELECT 1 FROM approver_permissions '
        'WHERE tenant_id=? AND approver=? AND operation=?',
        (p.tenant_id, approver, p.operation)
    ).fetchone()
    if not permission: return GateDecision.DENY
    state = c.execute(
        'SELECT revision FROM config_state WHERE tenant_id=?',
        (p.tenant_id,)
    ).fetchone()
    if not state: return GateDecision.DENY
    if state[0] == p.revision:
        return GateDecision.ALLOW
    return GateDecision.REVALIDATION_REQUIRED

def execute(c,p,approval_id='A-001',trusted_now=1000):
    """Errors before commit: rollback and raise. Ambiguous commit errors: raise, reconcile externally.
    Do not assert UNKNOWN on any arbitrary sqlite3 exception: ambiguity must be evidenced.
    """
    c.execute('BEGIN IMMEDIATE')
    try:
```

```
        decision=gate(c,p,approval_id,trusted_now)
        if decision!=GateDecision.ALLOW:
            c.rollback(); return decision,None
        old = c.execute(
            'SELECT retry_limit,revision FROM config_state '
            'WHERE tenant_id=?', (p.tenant_id,)
        ).fetchone()
        u = c.execute(
            'UPDATE config_state SET retry_limit=?, '
            'revision=revision+1 '
            'WHERE tenant_id=? AND revision=?',
            (p.value, p.tenant_id, p.revision)
        )
        if u.rowcount!=1:
            c.rollback(); return GateDecision.REVALIDATION_REQUIRED,None
        u = c.execute(
            "UPDATE approvals SET status='CONSUMED' "
            "WHERE approval_id=? AND tenant_id=? "
            "AND status='APPROVED'",
            (approval_id, p.tenant_id)
        )
        if u.rowcount!=1:
            c.rollback(); return GateDecision.DENY,None
        c.execute(
            'INSERT INTO audit('
            'approval_id,tenant_id,proposal_id,digest,old_value,'
            'new_value,revision_before,revision_after,at,result) '
            'VALUES (?,?,?,?,?,?,?,?,?,?)',
            (approval_id,p.tenant_id,p.proposal_id,digest(p),
             old[0],p.value,old[1],old[1]+1,trusted_now,'SUCCEEDED')
        )
        c.commit()
        return GateDecision.ALLOW,ExecutionResult.SUCCEEDED
    except Exception:
        if c.in_transaction: c.rollback()
        raise

DEFAULT=Proposal('P-001','tenant-01','app-config.json','set_config','retry_limit',5,3)
```

مرز تضمین **Replay** در این مثال: یکتایی `approval_id` و تغییر وضعیت همان تأیید به `CONSUMED` از استفاده دوباره همان شناسه جلوگیری می‌کند؛ اما در این نمونه برای زوج `(tenant_id, proposal_id)` قید یکتایی وجود ندارد. بنابراین جلوگیری از اجرای مجدد یک پیشنهاد با تأییدهای متفاوت، از این ۱۱ آزمون نتیجه نمی‌شود. در طراحی واقعی باید سیاست صدور تأیید، کلید یکتای مناسب و آزمون تأیید تکراری مشخص شود.

مرز تراکنش و اعتماد: `BEGIN IMMEDIATE`، مصرف تأیید، به‌روزرسانی وضعیت و ثبت Audit را فقط در همان پایگاه SQLite هماهنگ می‌کند؛ فایل `app-config.json` در این مثال تغییر نمی‌کند. هویت Approver، مجوزها و ساعت `trusted_now` ورودی‌های مورداعتماد برنامه فرض شده‌اند؛ احراز هویت، استقلال دو تأییدکننده، امنیت تغییر Policy و Audit ضددستکاری در این کد اثبات نشده‌اند.

حدود شواهد و سناریوی پیشرفته: یازده آزمون موجود عمدتاً اجرای ترتیبی را می‌سنجند، نه رقابت هم‌زمان دو Executor، دو Approval برای یک Proposal، مرز دقیق انقضا یا وضعیت نامعلوم پس از Commit. در خطای مبهم Commit، این نمونه `UNKNOWN` برنمی‌گرداند؛ خطا را بالا می‌فرستد و نیازمند Reconciliation بیرونی است. از فراگیر بخواهید برای این چهار مورد آزمون منفی و معیار قبولی طراحی کند؛ آن‌ها را «تست‌شده» معرفی نکنید.

شواهد آزمون: Valid، Tampered، Stale، Replay، Tenant mismatch، Expired، Unauthorized approver، Rollback هنگام خطای Allowlist، Bool rejection، Audit و Digest change: ۱۱/۱۱ موفق. جزئیات در `test_report.txt`؛ این ادعا درباره مخزن GitHub نیست.

بخش 17 از ۲۴ — مرز تصمیم: چرا **ALLOW** به‌معنای اجرا نیست؟

هدف: فراگیر بتواند با یک درخواست ثابت، ترتیب بررسی‌های Gate را توضیح دهد و سه خروجی DENY، REVALIDATION_REQUIRED و ALLOW را تشخیص دهد.

در مثال فرضی، Agent پیشنهاد P-001 را برای tenant-01 ثبت می‌کند: مقدار `retry_limit` از ۳ به ۵ و Revision مورد انتظار ۳. این پیشنهاد، مجوز نیست. تابع زیر فقط منطق تصمیم را توصیف می‌کند و هیچ عملیات واقعی روی فایل یا پایگاه داده انجام نمی‌دهد.

PSEUDOCODE — NOT EXECUTABLE — NOT AN EXECUTOR — COURSE-AUTHORED:

```
function evaluate(proposal, approval, trusted_authority, trusted_now, current_revision):
    if exact_schema_invalid(proposal): return DENY
    if policy_invalid(proposal) or time_invalid(trusted_now): return DENY
    if approval_missing_or_not_approved(approval): return DENY
    if digest_mismatch(proposal, approval): return DENY
    if tenant_or_scope_mismatch(proposal, approval): return DENY
    if expired(approval, trusted_now) or not trusted_authority: return DENY
    if current_revision != proposal.revision: return REVALIDATION_REQUIRED
    return ALLOW
```

شرح خط‌به‌خط: ابتدا شکل داده و سیاست کنترل می‌شوند. سپس وجود و وضعیت تأیید، اتصال Digest، محدوده Tenant و عملیات، انقضا و اختیار تأییدکننده سنجیده می‌شوند. تنها بعد از آن، Revision فعلی با Revision پیشنهاد مقایسه می‌شود. اگر همه کنترل‌ها موفق باشند، `ALLOW` صرفاً اجازه تلاش برای اجراست؛ موفقیت تغییر هنوز اثبات نشده است.

پرسش مدرس: «اگر مقدار پیشنهادی همان ۵ باشد ولی Revision فعلی ۴ شده باشد، چرا خروجی DENY نیست؟» پاسخ آموزشی: با فرض عبور از کنترل‌های قبلی، تغییر وضعیت موجب `REVALIDATION_REQUIRED` می‌شود.

محدودیت منبع: این شبه‌کد پیاده‌سازی TypeScript نیست؛ Runtime، Serialization، Validator دقیق Digest و اتصال به Trusted Store در آن نمایش داده نشده‌اند. تراکنش اتمیک TypeScript، مصرف تأیید و

ثبت Audit نیز در این تابع پیاده‌سازی نشده‌اند؛ برابری اجرای Python و TypeScript اثبات نشده است.

کارگاه معماری — اتصال **Policy Gate** به سامانه **Brownfield**

مسئله: یک سرویس Backend موجود، تنظیم `retry_limit` را در جدول `service_config` نگه می‌دارد. یک Specialized Coding Agent برای کاهش خطا پیشنهاد می‌دهد مقدار را از ۳ به ۵ تغییر دهد. Agent فقط اجازه خواندن کد و تولید Proposal دارد؛ هیچ دسترسی مستقیمی به دیتابیس Production، Migration Runner یا کلید امضای Approval ندارد.

مرز اعتماد: خروجی Agent حتی اگر شامل عبارت «approved» باشد داده نامطمئن است. فقط Approval صادرشده از کانال مستقل و دارای اختیار، پس از تطبیق هویت تأییدکننده، Digest، Scope، Tenant، انقضا و Revision معتبر است. اجرای تغییر نیز باید در تراکنش مستقل و همراه با ثبت Audit انجام شود.

لایه در سامانه موجود	مسئولیت قابل آزمون	کنترل شکست
Specialized Coding Agent	تولید Proposal نسخه‌دار با diff محدود	بدون دسترسی نوشتن یا اجرای ابزار حساس
Proposal API	Canonicalize و محاسبه Digest از داده معتبر	رد ورودی ناشناخته و Schema ناسازگار
Approval Service	ثبت تصمیم شخص/سرویس مجاز مستقل از Agent	رد self-approval و Scope نامعتبر
Policy Gate	بررسی Tenant، Digest، Revision، Expiration، Replay و Allowlist	Fail closed؛ تصمیم ALLOW هنوز اجرا نیست
Transactional Executor	مصرف یک‌باره Approval، compare-and-swap و Audit در تراکنش	Rollback در خطا؛ عدم اعمال تغییر جزئی

نمونه قرارداد آموزشی **(PSEUDOCODE — NOT EXECUTABLE)**:
این قطعه، طرح ادغام است و جایگزین فایل اجرایی
`session01_reference.py` یا آزمون‌های واقعی آن نیست.

```
proposal = proposal_api.validate_and_digest(agent_output)
approval = approval_store.load(proposal.approval_id)  # independent channel
with db.transaction(immediate=True):
    gate.assert_authorized(proposal, approval, current_revision)
    approval_store.consume_once(approval.id)
    config_repo.compare_and_swap(
        tenant=proposal.tenant_id,
        key="retry_limit",
        expected_revision=proposal.expected_revision,
        new_value=5,
    )
    audit_repo.append_execution(proposal.digest, approval.id)
# COMMIT only if every operation succeeds; exceptions must propagate
```

سناریوهای آزمون طراحی: (۱) Tenant متفاوت → DENY؛ (۲) تغییر مقدار پس از Digest mismatch → Approval؛ (۳) Revision کهنه → Conflict؛ (۴) مصرف دوباره Approval → Replay denied؛ (۵) خطای ثبت Audit → Rollback همه تغییرات؛ (۶) متن «approved» در پاسخ Agent بدون Approval مستقل → DENY.

خروجی قابل سنجش: یک نمودار Sequence پنج‌لایه، یک ADR کوتاه درباره محل اجرای Gate، یک Threat/Control Matrix با شش سناریو و برنامه مهاجرت بدون شکستن API موجود (Feature Flag، آزمون رگرسیون، Shadow Evaluation و Rollback). معیار قبولی: نبود مسیر اجرای مستقیم از Agent به Executor و پوشش هر شش آزمون منفی.

محدودیت: مثال فوق معماری آموزشی است؛ صحت در محیط Production نیازمند بررسی تراکنش‌های واقعی، هم‌زمانی، idempotency، سطح دسترسی و استقرار است.

بخش 18 از ۲۴ — سه سناریوی قابل تدریس و نمونه واقعی قابلیت **Agent**

CONCEPTUAL CLASSROOM DEMONSTRATIONS — NOT TEST EXECUTION RESULTS

01 **CASE /** تأیید معتبر: P-001 برای tenant-01، مقدار ۵، Revision ۳، Digest منطبق، تأیید معتبر و اختیار فعال دارد. نتیجه مورد انتظار Gate برابر `ALLOW` است. تنها پس از موفقیت تراکنش محلی می‌توان `SUCCEEDED` گزارش کرد.

02 **CASE /** دست‌کاری پیشنهاد: تأیید برای مقدار ۵ باقی می‌ماند اما مقدار ارسالی ۹ می‌شود. Digest دیگر منطبق نیست؛ نتیجه مورد انتظار `DENY` و وضعیت بدون تغییر است.

CASE 03 **/ Replay**: یک تأیید یک‌بارمصرف پس از استفاده مجدد ارائه می‌شود. نتیجه مورد انتظار تلاش دوم `DENY` است و Revision فقط یک بار افزایش می‌یابد.

VERIFIED SOURCE EXCERPT — تطبیق با فایل و Blob در Commit مشخص؛ وضعیت مجوز مخزن در انتظار بررسی.

`AlinaMatrix Enterprise — packages/domain/src/agents.ts`

```
export const AGENT_CAPABILITIES: Readonly<Record<AgentName, ReadonlySet<AgentCapability>>> = {
  AlinaArchitect: new Set(["READ_SOURCE", "READ_EVIDENCE", "WRITE_DRAFT"]),
  AlinaGuard:     new Set(["READ_EVIDENCE", "RUN_VALIDATION"]),
  AlinaDocEngine: new Set(["COMPILE_ARTIFACT"]),
  AlinaOptimizer: new Set(["READ_USAGE", "WRITE_CACHE_METADATA"]),
};
```

Source commit URL as printed in the original PDF: `https://github.com/aminbita162-glitch/AlinaMatrix-Enterprise/blob/`

47eb2c2a108f91bd2c8d3f0311a79d30cec26971/packages/domain/src/agents.ts

:Blob SHA verified against GitHub
ca3ac3ce6672a0590a4198264aac0a298fbd3cb5

توضیح مدرس: READ_SOURCE و WRITE_DRAFT قابلیت‌های Agent هستند، نه اختیار انسانی برای تأیید Release یا اجرای تغییر. انواع AgentName و AgentCapability در بخش دیگری از کد تعریف شده‌اند.

بخش 19 از ۲۴ — شواهد چه چیزی را ثابت می‌کنند؟

هدف: فراگیر میان مشاهده، ادعای مجاز و ادعای اثبات‌نشده تفاوت بگذارد.

ادعا	ارزیابی و محدوده
نمونه محدود کلاسی، اتصال تأیید به پیشنهاد و کنترل Policy و Revision را نشان می‌دهد	ادعای آموزشی محدود و منطبق با متن منبع؛ مشروط به همان فرض‌ها
مصرف تأیید در تراکنش محلی SQLite نمایش داده شده است	فقط محدوده SQLite محلی، نه همه آثار جانبی خارجی
سیستم از نظر رمزنگاری احراز هویت شده است	اثبات نشده؛ ادعا نشود
فایل واقعی به‌صورت اتمیک بازنویسی شده است	اثبات نشده؛ ادعا نشود
رفتار اجرایی Python و TypeScript یکسان دارند	اثبات نشده؛ ادعا نشود
	اثبات نشده؛ ادعا نشود

ادعا	ارزیابی و محدوده
Crash Recovery آزموده و تأیید شده است	

شواهد مستقلِ لازم: Identity Assurance، مدیریت اختیارات، رقابت چندفرایندی، دوام تراکنش، نگهداری Audit، آثار توزیع‌شده و حل وضعیت UNKNOWN، هر کدام ارزیابی و شواهد خاص خود را می‌خواهند.

پرسش مدرس: «اگر یک تست SQLite موفق شود، آیا می‌توانیم بگوییم تغییر فایل در سیستم‌عامل هم اتمیک است؟» پاسخ: خیر؛ این دو محدوده آزمون متفاوت‌اند.

جدول تصمیم مدرس — سناریوهای ثابت

این جدول برای مرور ۹۰ دقیقه‌ای است؛ خروجی اجرا فقط در سناریوی معتبر و در صورت موفقیت اجرای تراکنش «SUCCEEDED» است.

سناریو	Gate	تغییر وضعیت	نتیجه اجرا	علت
Valid	ALLOW	بله، در اجرای موفق	SUCCEEDED	Digest و Policy و Revision معتبر
Tampered: 9	DENY	خیر	اجرا نمی‌شود	عدم تطابق Digest؛ ۹ در محدوده Policy است
Stale: revision 4	REVALIDATION_REQUIRED	خیر	اجرا نمی‌شود	Revision مبنای تأیید قدیمی است
Replay	DENY	خیر	اجرا نمی‌شود	تأیید قبلاً CONSUMED شده
Tenant mismatch	DENY	خیر	اجرا نمی‌شود	محدوده Tenant همخوان نیست
Expired	DENY	خیر	اجرا نمی‌شود	زمان معتبر گذشته است
Unauthorized approver	DENY	خیر	اجرا نمی‌شود	مرجع اختیار اجازه نمی‌دهد

بخش 20 از ۲۴ — تمرین کاغذی ۱۵ دقیقه‌ای، بدون نیاز به دانلود

CLASSROOM FIXTURE — HYPOTHETICAL / PAPER-BASED — NO DOWNLOAD REQUIRED

وضعیت اولیه: Tenant برابر `tenant-01`، هدف منطقی `app-config.json`، مقدار `retry_limit=3` و Revision برابر ۳ است. Proposal با شناسه `P-001` درخواست `set_config` برای کلید `retry_limit` با مقدار ۵ و Revision مورد انتظار ۳ دارد. Approval با شناسه `A-001` در مخزن مورد اعتماد موجود است: شناسه Proposal و Tenant یکسان‌اند؛ Operation و Target منطبق‌اند؛ Digest ذخیره‌شده مربوط به پیشنهاد اصلی با مقدار ۵ است؛ وضعیت `APPROVED`، تأیید منقضی‌نشده، اختیار تأییدکننده فعال و تأیید یک‌بارمصرف است.

تمرین ۵ — **A** دقیقه: سه مرحله Proposal، Approval و Execution را مشخص کنید. فیلدهایی را که باید به تأیید متصل باشند زیرخط بکشید.

۵ — **Task B** دقیقه: برای پیشنهاد اصلی یکی از `ALLOW`، `DENY` یا `REVALIDATION_REQUIRED` را انتخاب کنید و توضیح دهید چرا `ALLOW` هنوز `SUCCEEDED` نیست.

۵ — **Task C** دقیقه: فقط مقدار ارسالی Proposal را از ۵ به ۹ تغییر دهید و Approval ذخیره‌شده را ثابت نگه دارید. تصمیم Gate و تنها کنترلی را که این تغییر را رد می‌کند نام ببرید.

راهنمای مدرس: پاسخ‌ها و معیار نمره‌دهی در بخش محرمانه Instructor Answer Key باقی می‌مانند؛ در برگه فراگیر افشا نشوند.

بخش 21 از ۲۴ — پانزده دقیقه دوم تمرین عملی

هدف آموزشی: فراگیر تصمیم Gate را از نتیجه اجرای واقعی تفکیک کند و بتواند Replay و Revision drift را توضیح دهد.

۵ دقیقه | **Task D — HYPOTHETICAL**: پیشنهاد اصلی P-001 با مقدار ۵ و Revision مورد انتظار ۳ را بازگردانید، ولی Revision فعلی را ۴ قرار دهید. ابتدا نتیجه Gate را حدس بزنید؛ سپس توضیح دهید چرا تأیید قدیمی بعد از تغییر Revision خودکار قابل استفاده نیست. مدرس قبل از نمایش پاسخ، دلیل هر فراگیر را بشنود.

۵ دقیقه | **Task E — HYPOTHETICAL**: Revision را ۳ فرض کنید و تصور کنید نخستین تراکنش مجاز محلی با Commit موفق، مقدار `retry_limit` را ۵ کرده، Revision را به ۴ رسانده و تأیید A-001 را `CONSUMED` کرده است. همان تأیید دوباره ارسال می‌شود. نتیجه و امکان تغییر دوم را تحلیل کنید. نکته آموزشی: وضعیت مصرف‌شده قبل از اختلاف Revision بررسی می‌شود.

۵ دقیقه: جدول سه‌ردیفی «Original / Tampered / Replay» بسازید. برای هر ردیف بنویسید: تصمیم Gate، آیا وضعیت تغییر می‌کند، و علت. سپس توضیح دهید چرا مصرف تأیید و تغییر وضعیت حفاظت‌شده باید در یک تراکنش محلی باشند و دو محدودیت مثال را نام ببرید. **Task F —**

تحویل تمرین: یک جدول حاشیه‌نویسی‌شده و توضیح کوتاه درباره Atomicity؛ برای اجرای تمرین کاغذی، دانلود کد، اتصال به سامانه واقعی یا ایجاد Side Effect خارجی لازم نیست. توسعه کدنویسی فقط با Fixture کامل و تأییدشده مدرس ممکن است.

Task G — توسعه اختیاری مدرس، ۵ دقیقه، خارج از تمرین اجباری ۳۰ دقیقه‌ای: پس از رسیدن اجرای تراکنش به `commit()` ارتباط پیش از دریافت تأیید قطع می‌شود. نتیجه امن گزارش چیست؟ چرا Retry خودکار بدون Reconciliation و Idempotency ناامن است؟

بخش 22 از ۲۴ — پاسخ‌نامه محرمانه و شیوه نمره‌دهی

پاسخ‌های مرجع، منطق نمره‌دهی و کلید سناریوها فقط در سند مستقل و محرمانه Session01_Answer_Key ارائه می‌شوند. این راهنما شامل پاسخ تمرین‌ها نیست. نسخه فراگیر باید جداگانه و بدون پاسخ توزیع شود.

بخش 23 از ۲۴ — جمع‌بندی، منابع و جلسه بعد

شناسنامه منشأ سه قطعه کد — کنترل **GitHub** در ۱۴ اکتبر ۲۰۲۶

منبع — AEGIS — core/human_authority.py

Commit — `4f310113c49c0ea17c865a5af2670c0a01707655`

Blob SHA — `8e3c6a4defe0b4263ccc8cba1ef87435eee26877`

مجوز / وضعیت — مالک پروژه، استفاده آموزشی را مجاز اعلام کرده است؛ متن LICENSE مخزن حاوی یادداشت ویراستاری است و نباید بدون بازبینی به‌عنوان مجوز نهایی بازنشر تلقی شود.

منبع — AEGIS — core/authority_matrix.py

Commit — `f845ab702fbafe1b6b6ea2abca842de50c7bb206`

Blob SHA — `9409ecd6046e6c3d03026c9f01c05fe57a05e7fc`

مجوز / وضعیت — مالک پروژه، استفاده آموزشی را مجاز اعلام کرده است؛ متن LICENSE مخزن حاوی یادداشت ویراستاری است و نباید بدون بازبینی به‌عنوان مجوز نهایی بازنشر تلقی شود.

منبع — AlinaMatrix — packages/domain/src/agents.ts

Commit — `47eb2c2a108f91bd2c8d3f0311a79d30cec26971`

Blob SHA — `ca3ac3ce6672a0590a4198264aac0a298fbd3cb5`

مجوز / وضعیت — مالک پروژه، استفاده از قطعه در این دوره را مجاز اعلام کرده است؛ فایل LICENSE در ریشه مخزن در بررسی مستقل پیدا نشد. این اجازه، جایگزین متن مجوز عمومی مخزن نیست.

تأیید Blob به معنی تأیید مجوز بازنشر یا تأیید امنیت کل پروژه نیست. برای استفاده خارج از این دوره، وضعیت مجوز باید مستقل بررسی شود.

پنج اصل کلیدی: (۱) Proposal مرجع اختیار نیست. (۲) Approval باید به داده دقیق و Scope متصل باشد. (۳) اختیار در لحظه Execution دوباره بررسی می‌شود. (۴) تصمیم Gate از نتیجه Execution جداست. (۵) Approval یک‌بارمصرف همراه با تغییر وضعیت حفاظت‌شده مصرف می‌شود.

مرجع معماری آموزشی: AEGIS AGENT و AlinaMatrix Enterprise. هر سه فایل در GitHub در Commit مشخص بازخوانی و شناسه Blob آن‌ها تطبیق داده شد. وضعیت مجوز مخزن AlinaMatrix هنوز روشن نیست؛ کد آموزشی و شبه‌کد از نقل مستقیم مخزن جدا هستند.

مراجع رسمی:

https://docs.python.org/3/library/sqlite3.html
https://docs.python.org/3/library/json.html
https://docs.python.org/3/library/hashlib.html
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/isSafeInteger

جلسه بعد: `Session 02 — Proposal Contract and Structured Output`.

کنترل پایانی مدرس پیش از پایان جلسه: از هر فراگیر بخواهید یک مسیر تصمیم را با دلیل توضیح دهد، نه فقط نام وضعیت را تکرار کند.

در پاسخ، باید اتصال Approval به Digest و Revision و Scope روشن باشد؛ همچنین فراگیر توضیح دهد که `ALLOW` تنها مجوز ورود به مرحله اجراست و موفقیت اجرای واقعی را ثابت نمی‌کند.

برای سناریوی Replay، از فراگیر بخواهید تفاوت «مصرف یک Approval» و «جلوگیری از اجرای دوباره Proposal» را بیان کند. اگر

پاسخ فاقد مرز تراکنش SQLite و اثر جانبی خارجی بود، همان‌جا اصلاح مفهومی انجام دهید.

بخش 24 از ۲۴ — مالکیت، مجوز تدریس و منشأ محتوا

LICENSE / PROVENANCE — متن حقوقی اصلی به انگلیسی برای جلوگیری از تغییر ناخواسته دامنه مجوز حفظ شده است.

Copyright © 2026 Amin Azimi. All rights reserved.

Author: Amin Azimi — AI Architect — End-to-End System Development. Brand: Azimi Innovation Lab.

Named instructor teaching permission: Mozhgan Etaati (مژگان اعطایی) and Amirhossein Enayati (امیرحسین عنایتی), software engineers (مهندسان نرم‌افزار), alone are authorized to study this instructor-only document and use it to prepare and deliver online or in-person lessons. They may explain its concepts in their own words and prepare their own teaching materials. Learners must not receive this document or copies of it, whether digital or printed. The instructor answer key must not be handed out. Attribution and this license page must remain intact.

Restrictions: No distribution of the PDF or other digital document copies, public resale, republication as a standalone product, removal of author attribution, or sublicensing without written permission from the copyright owner. Code examples are educational adaptations and carry no guarantee of fitness for production deployment. Third-party code, logos and linked resources remain subject to their respective rights and licenses.

Provenance statement: Only the three boxes explicitly marked VERIFIED SOURCE EXCERPT reproduce excerpts from the linked GitHub repositories. All other code is course-authored educational adaptation or non-executable pseudocode. Third-party excerpts remain subject to their respective rights and licenses.

Contact for permission requests: azimiamin3333@yahoo.com. Translation, redistribution, recording and publication of course material require separate written authorization from the copyright holder.